

DEMAND-008：目录框架重构与实习清单融合

万能产品小助手 (PM AI)
魏子政审阅

2026-05-06 / 版本 v0.1

提出日期	2026-05-06
提出者	魏子政
优先级	P0
状态	进行中
总需求编号	D-014
前置需求	D-011 (MVP)、D-012 (前端重写)、D-013 (流转修复)
参考文档	docs/intern-dev-checklist.tex (实习生开发清单)

需求背景

魏子政要求将 MVP 阶段生成的代码清空,重新编排前后端目录框架。同时要求将 docs/intern-dev-checklist.tex (实习生开发清单) 中的工程规范与现有 MASTER 总需求做融合,形成正式的、面向多人协作的目录结构。

核心目标：

- 1. 清空 MVP 代码,保留配置文件和目录骨架
- 2. 基于实习清单的完整工程视角,重新设计前后端目录结构
- 3. 将目录结构与 MASTER 需求做映射,明确每个目录/文件对应哪个需求
- 4. 为后续 Cursor 代码生成提供清晰的目录蓝图

实习清单核心内容提取

intern-dev-checklist.tex 是一份面向实习生的 14 章开发指南,覆盖以下领域：

前端目录框架

设计原则

- **admin / portal 分离**：管理后台与开发者门户分目录,职责清晰
- **composables**：可复用逻辑抽离 (如 useAuth、usePagination)

章 主题	与 MASTER 需求的映射
1 环境准备	D-008 (开发环境搭建)
2 前端开发 (结构 + 任务 + 规范)	D-012 (前端重写)
3 后端开发 (结构 + 任务 + 规范)	D-011 (MVP 后端)
4 数据库	D-011 (PostgreSQL + Alembic)
5 搜索引擎	D-015 (搜索, 待建)
6 安全扫描	D-011 (三层扫描已完成)
7 异步任务	D-011 (BackgroundTasks)
8 对象存储	D-011 (MinIO)
9 认证授权	D-011 (JWT)
10 容器化部署	D-016 (生产部署, 待建)
11 日志监控	D-017 (可选项)
12 开发路线图	整体迭代规划
13 企业级演进方向	长期架构升级计划
14 名词速查表	团队知识库

- **API 层统一**: 所有接口调用集中在 `src/api/`, 组件不直接发请求
- **TypeScript 优先**: 所有 `.vue` 和 `.ts` 文件, 不用 `.js`

目录结构

frontend/

```
|-- src/
|   |-- views/                <- 页面文件
|   |   |-- admin/           <- 管理后台
|   |   |   |-- apps/        <- 应用管理 (上架/下架/删除)
|   |   |   |-- reviews/     <- 审批管理 (审批工作台)
|   |   |   |-- users/       <- 用户管理 (RBAC 角色分配)
|   |   |-- portal/          <- 开发者门户
|   |   |-- explore/         <- 应用市场/发现页
|   |   |-- submit/          <- 提交应用页
|   |   |-- profile/         <- 开发者主页 (我的应用)
|   |   |-- LoginView.vue    <- 登录页
|   |   |-- RegisterView.vue <- 注册页
|   |-- components/          <- 可复用组件
|   |   |-- ui/               <- 基础 UI 组件 (Element Plus 二次封装)
|   |   |-- AppCard.vue       <- 应用卡片 (市场列表用)
|   |   |-- SearchBar.vue     <- 搜索栏 (带防抖)
|   |   |-- ScanResult.vue    <- 扫描结果展示
|   |   |-- StatusTag.vue     <- 状态标签 (scanning/pending/published/rejected)
```

```

| |-- composables/          <- Vue 3 Composables (可复用逻辑)
| |   |-- useAuth.ts        <- 认证状态
| |   |-- usePagination.ts  <- 分页逻辑
| |   |-- useSearch.ts      <- 搜索防抖
| |-- stores/               <- Pinia 状态管理
| |   |-- auth.ts           <- 用户认证 store
| |   |-- skills.ts         <- Skill 数据 store
| |   |-- ui.ts             <- UI 状态 (侧边栏、暗色模式等)
| |-- api/                  <- API 调用封装 (axios)
| |   |-- client.ts         <- axios 实例 + 拦截器
| |   |-- auth.ts           <- 登录/注册/Token 刷新
| |   |-- skills.ts         <- Skill CRUD + 搜索 + 下载 + 安装
| |   |-- reviews.ts       <- 审批接口
| |   |-- users.ts          <- 用户管理接口
| |   |-- workflow.ts       <- 工作流/状态流转
| |   |-- types.ts          <- 全局 TypeScript 类型定义
| |-- router/               <- Vue Router 路由配置
| |   |-- index.ts          <- 路由定义 + 路由守卫
| |-- utils/                <- 工具函数
| |   |-- jwt.ts            <- JWT 解析
| |   |-- format.ts         <- 日期/文件大小格式化
| |-- styles/               <- 全局样式
| |   |-- variables.css     <- CSS 变量 (主题色)
| |   |-- global.css        <- 全局样式重置
| |   |-- element-override.css <- Element Plus 主题覆盖
| |-- App.vue               <- 根组件
| |-- main.ts               <- 入口文件
|-- public/
| |-- icons/                <- 静态图标
|-- index.html
|-- vite.config.ts
|-- tsconfig.json
|-- package.json

```

目录与需求映射

后端目录框架

设计原则

- **core/**: 核心配置与安全模块, 独立于业务

目录/文件	对应需求	说明
views/portal/explore/	D-015 (搜索)	应用市场 + 搜索 + 筛选
views/portal/submit/	D-011	Skill 上传表单
views/portal/profile/	D-013	我的应用 (所有状态)
views/admin/apps/	D-011	管理员应用管理
views/admin/reviews/	D-011	审批工作台
views/admin/users/	D-018 (RBAC)	用户管理 + 角色分配
components/ScanResult.vue	D-011	三层扫描结果展示
composables/useSearch.ts	D-015	搜索防抖 + OpenSearch 对接
api/skills.ts	D-013	下载 + 安装接口

- **models/**: SQLAlchemy ORM 模型, 每个实体一个文件
- **routers/**: API 路由按功能分文件, 每个文件一个 **APIRouter**
- **services/**: 业务逻辑层, Router 不直接操作数据库
- **schemas/**: Pydantic 数据模型, 请求/响应格式定义
- **tasks/**: asyncio 后台任务
- **tests/**: 单元测试 + 集成测试

目录结构

```

backend/
|-- app/
|   |-- main.py           <- FastAPI 入口, 注册路由 + 启动事件
|   |-- config.py        <- 配置 (环境变量, Pydantic Settings)
|   |-- database.py      <- 数据库连接 + Session 工厂
|   |-- core/            <- 核心模块
|       |-- security.py  <- JWT 签发/验证 + 密码哈希
|       |-- deps.py      <- FastAPI 通用依赖 (get_db, get_current_user)
|       |-- permissions.py <- RBAC 权限装饰器
|   |-- models/          <- SQLAlchemy ORM 模型
|       |-- base.py      <- Base + 公共字段 (id, created_at, updated_at)
|       |-- user.py      <- 用户表
|       |-- skill.py      <- Skill 表
|       |-- skill_version.py <- Skill 版本表
|       |-- scan_result.py <- 扫描结果表
|       |-- review.py    <- 审批记录表
|   |-- schemas/         <- Pydantic 数据模型
|       |-- auth.py      <- 登录/注册请求响应
|       |-- skill.py     <- Skill CRUD 请求响应
|       |-- review.py    <- 审批请求响应

```

```

|   |   |-- common.py           <- 分页、通用响应
|   |-- routers/                <- API 路由
|   |   |-- auth.py             <- /api/auth/*
|   |   |-- skills.py           <- /api/skills/*
|   |   |-- reviews.py          <- /api/reviews/*
|   |   |-- search.py           <- /api/search/* (OpenSearch 对接)
|   |   |-- users.py            <- /api/users/* (RBAC 管理)
|   |   |-- workflow.py         <- /api/workflow/* (状态流转)
|   |-- services/               <- 业务逻辑层
|   |   |-- auth_service.py     <- 认证业务逻辑
|   |   |-- skill_service.py    <- Skill CRUD 逻辑
|   |   |-- scan_service.py     <- 三层扫描编排
|   |   |-- semgrep_scan.py     <- Semgrep 扫描
|   |   |-- clamav_scan.py      <- ClamAV 扫描
|   |   |-- llm_scan.py         <- LLM 语义分析
|   |   |-- search_service.py   <- OpenSearch 搜索逻辑
|   |   |-- skill_package_service.py <- 安装包管理 (下载/安装到本地)
|   |   |-- seed.py             <- 种子数据
|   |-- tasks/                  <- 后台任务
|   |   |-- scan_task.py        <- 扫描任务触发
|   |   |-- notify_task.py      <- 通知任务 (预留)
|   |-- utils/                  <- 工具函数
|   |   |-- minio_client.py     <- MinIO 对象存储客户端
|   |   |-- password.py         <- 密码哈希
|   |   |-- opensearch.py       <- OpenSearch 客户端
|-- tests/
|   |-- unit/                   <- 单元测试
|   |-- integration/            <- 集成测试
|-- alembic/                    <- 数据库迁移
|   |-- versions/
|-- requirements.txt
|-- Dockerfile
|-- .env                        <- 环境变量 (不提交 Git)

```

目录与需求映射

编码规范 (融合实习清单)

前端编码规范

1. 组件用 TypeScript, 文件名 PascalCase (如 AppCard.vue)

目录/文件	对应需求	说明
core/security.py	D-011	JWT + 密码哈希
core/permissions.py	D-018 (RBAC)	角色权限控制
models/	D-008	所有数据库实体
routers/search.py	D-015	OpenSearch 搜索接口
services/search_service.py	D-015	搜索业务逻辑
services/scan_*	D-011	三层扫描 (已完成)
services/skill_package_service.py	D-013	下载 + 安装到 OpenClaw
tasks/scan_task.py	D-011	后台扫描任务
utils/opensearch.py	D-015	OpenSearch 客户端封装

2. 遵循 `<script setup lang="ts">` 风格
3. UI 组件优先用 Element Plus, 复杂交互可二次封装
4. API 调用统一放 `src/api/`, 用 axios 封装, 不在组件里直接 `fetch`
5. 状态管理用 Pinia, 按功能模块分 store
6. 可复用逻辑抽到 `composables/`
7. 样式用 CSS 变量 + Element Plus 主题覆盖
8. 颜色方案: 黑白灰极简 (主色 #1a1a2e, 辅助 #f5f5f5, 文字 #333333)

后端编码规范

1. 所有接口用 `async/await`
2. 数据校验用 Pydantic, 不手动校验
3. 数据库操作放 `services/`, 不在 Router 里直接操作
4. 敏感信息放环境变量, 不硬编码
5. 每个 Router 加 `tags`, 方便 Swagger 文档
6. 每个模块有对应的 `tests/unit/` 和 `tests/integration/` 测试

迭代规划 (融合实习清单路线图)

企业级演进方向 (引用实习清单第 13 章)

以下为 MVP 后的升级计划, 不做当前迭代, 但目录结构需预留扩展空间:

验收标准

- ☐ 前端目录结构符合上述规范
- ☐ 后端目录结构符合上述规范
- ☐ MVP 旧代码已清空 (仅保留 `.gitkeep` 和 `__init__.py`)

阶段	重点	对应需求
V1.1	目录框架重构 + 环境重建 清空旧代码，按新目录结构重建 安装依赖，确认服务可启动	D-014（本文档）
V1.2	核心功能重建 Skill CRUD + 文件上传 + MinIO 三层扫描 + 审批工作台 登录/注册 + JWT 应用市场 + 详情页 + 我的应用 下载 zip + 安装到 OpenClaw	D-011 + D-012 + D-013
V1.3	搜索 + RBAC OpenSearch 全文 + 语义搜索 RBAC 四角色权限体系 用户管理页面	D-015 + D-018
V1.4	打磨 + 部署 UI 优化 + 响应式 Docker 生产配置 健康检查 + 日志	D-016

- ☐ npm install + npm run build 无报错（即使页面为空）
- ☐ 后端 pip install + uvicorn 可启动（/health 正常）
- ☐ 需求文档 MASTER 已同步更新
- ☐ DOC-NAMING-CONVENTION 索引已更新

更新记录

版本	日期	变更内容
v0.1	2026-05-06	初始版本

维度	现在 (MVP)	未来 (企业级)
认证	自建 JWT	Keycloak (LDAP/SSO)
工作流	数据库状态机	Temporal (分布式工作流)
消息	asyncio	Kafka (消息队列)
存储	PostgreSQL JSONB	PostgreSQL + MongoDB
部署	Docker Compose	Kubernetes + Helm
监控	structlog	Prometheus + Grafana + Loki
前端	Element Plus	自研组件库